

Хабр

Весна новых возможностей

Свежие IT-события уже в Хабре. Каждый день.

Хабр



Чему научиться в этом году?



vladipirogov

16 часов назад

От PLC к своему HMI и AI-анализу



Простой



12 мин



6.4K

DIY или Сделай сам, Прототипирование*, Программирование микроконтроллеров*, Облачные сервисы*, Искусственный интеллект*

Тutorial

В [первой статье](#) и [второй статье](#) мы заставили крутиться PMSM-мотор под управлением комплекта P-NUCLEO-IHM03, внедрив в прошивку поддержку FDCAN. Чтобы превратить DIY-проект в нечто более серьезное, нам нужен верхний уровень: наглядный интерфейс, интеграция с промышленными стандартами и, конечно, немного магии современных LLM для диагностики.

Сегодня мы построим распределенную систему управления, где OpenPLC берет на себя логику, Node-RED — визуализацию, а AI-агент — роль инженера-диагноста.

HMI на стероидах: Когда консоли уже мало

Просто видеть сырые данные в терминале — сомнительное удовольствие. Для управления и мониторинга привода нам нужен Dashboard, который легко использовать. Node-RED здесь хорошо подходит: он легкий, живет на Orange Pi и имеет нативную поддержку всего, что нам нужно.

Мы не будем просто рисовать графики. Наша задача — создать мост между CAN bus, Modbus и миром IP-протоколов.

Установка и настройка Node-RED на Orange Pi

Node-RED — это визуальная IoT-платформа с потоковой моделью программирования. В нашем проекте она выполняет сразу несколько ролей: читает фреймы с CAN bus через SocketCAN, опрашивает регистры OpenPLC по Modbus TCP, отображает всё это в HMI-дашборде и запускает анализ телеметрии через AI-агент. Альтернативы — Grafana с плагинами или написанный с нуля веб-интерфейс — при тех же задачах потребовали бы значительно больше усилий.

Mosquitto — легковесный MQTT-брокер. В проекте он служит шиной сообщений между Node-RED и n8n: Node-RED публикует подготовленную телеметрию в топик `node-red/request`, n8n подписывается на него, выполняет RAG + LLM анализ и возвращает диагноз обратно в топик `node-red/data`. MQTT здесь удобнее прямого HTTP-вызова: n8n и Node-RED развязаны асинхронно и не зависят от порядка запуска.

Оба инструмента хорошо документированы. Для установки на Orange Pi (Debian/Ubuntu-based) достаточно официальных инструкций:

- **Node-RED** — [Running on Raspberry Pi / Orange Pi](#) (скрипт-установщик, настройка systemd-сервиса)
- **Mosquitto** — [официальная документация](#) и [руководство по установке на Debian/Ubuntu](#)

Автоматический запуск интерфейса CAN bus

Для того чтобы интерфейс `can0` поднимался автоматически при загрузке системы на Orange Pi, лучше всего использовать **systemd** или файл конфигурации сетевых интерфейсов.

1. Создайте файл конфигурации:

```
sudo nano /etc/systemd/network/80-can0.network
```

[Объяснить с](#)  SourceCraft

2. Вставьте следующее содержимое:

```
[Match]
Name=can0

[CAN]
BitRate=500K
```

[Объяснить с](#) 

Примечание: `500K` эквивалентно скорости `500000`. Скорость CAN bus 500 kbit/sec ранее настраивалась для платы управления PMSM.

3. Включите службу:

```
sudo systemctl enable systemd-networkd
sudo systemctl restart systemd-networkd
```

[Объяснить с](#) 

Для ручного запуска интерфейса CAN bus (если автоматический запуск не настроен) необходимо выполнить команду

```
sudo ip link set can0 up type can bitrate 500000
```

[Объяснить с](#) 

Установка необходимых плагинов Node-red

Напомним, что установка плагинов производится с помощью меню "Управление палитрой". Или можно установить в консоли командой `npm install`.

Для демо проекта нам необходимы плагины:

- [@flowfuse/node-red-dashboard](#) - компоненты для создания визуализации HMI
- [@flowfuse/node-red-dashboard-2-ui-led](#) - дополнительный компонент индикации для HMI
- [node-red-contrib-modbus](#) - Компоненты для работы с Modbus
- [node-red-contrib-socketcan](#) - Компоненты для работы с CAN bus

Установка Ollama, n8n и Qdrant через Docker Compose

Для работы AI-агента потребуются три сервиса: **Ollama** (локальный LLM-сервер с embedding-моделью), **n8n** (платформа для построения AI-агентов) и **Qdrant** (векторная база данных для RAG).

Удобнее всего поднять их вместе через Docker Compose.
Готовый файл [docker-compose.yml](#) находится в репозитории.

Модель эмбедингов qllama/bge-small-en-v1.5

В `entrypoint` сервиса Ollama автоматически скачивается модель `qllama/bge-small-en-v1.5` — компактная (~33 МБ) модель для вычисления векторных эмбедингов текста, оптимизированная для задач семантического поиска. Она используется в RAG-пайплайне: перед отправкой данных в Qdrant и при поиске релевантных документов в n8n-агенте.

Запуск

```
docker compose up -d
```

[Объяснить с](#) 

Сервисы будут доступны по адресам:

- **n8n:** `http://<IP>:5678`
- **Ollama API:** `http://<IP>:11434`
- **Qdrant:** `http://<IP>:6333`

При первом запуске Ollama автоматически загрузит модель `qllama/bge-small-en-v1.5`.
Проверить можно командой:

```
docker logs ollama
```

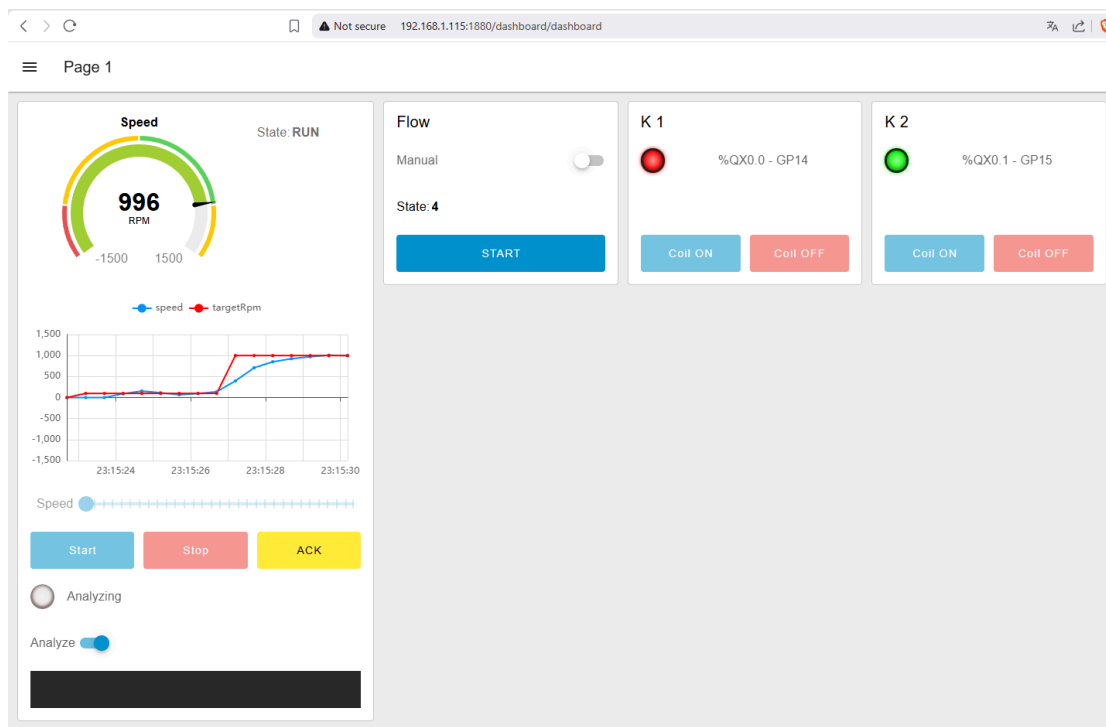
[Объяснить с](#) 

Описание HMI-интерфейса

Далее идет краткое описание каждой панели интерфейса и скрин соответствующей части логики в `node-red flow`. Содержимое раздела скорее похоже на справочный мануал, которые обычно скучный и пролистывается. Поэтому он приведен для понимания общей картины и как справочное описание. Ниже в разделе тестирования постараюсь наглядно показать, как это работает. А кто желает подробнее разобраться, может клонировать репозиторий и запустить проект.

Для открытия HMI-интерфейса необходимо набрать в строке браузера

```
http://<IP_ADDRESS>:1880/dashboard
```



Speed dashboard

Панель мониторинга и контроля работы двигателем PMSM.

Получение и отправка команд для двигателя происходит с помощью компонентов socketcan.

SocketCAN - это набор опенсорсных драйверов CAN-протокола и сетевой стек, реализованный в ядре Linux.

Если говорить проще, SocketCAN превращает CAN-интерфейс (который обычно работает как последовательный порт) в стандартное сетевое устройство, подобное Ethernet (eth0 или wlan0). Это позволяет работать с CAN bus через привычный программный интерфейс сокетов.

Верхняя группа индикаторов

- Вверху круговая диаграмма текущего значения скорости
- State - индикатор состояния работы двигателя (Состояния описаны в предыдущей статье)
- Ниже линейный график (Speed_chart) - сравнивает значение заданной скорости с текущей. Активный только при включенном слайдере Analyze

Группа ручного управления

- Ползунок Speed - ручное задание скорости вращения двигателя
- Кнопки:
 - Start - ручной запуск двигателя с минимальной скоростью
 - Stop - ручная остановка двигателя
 - ACK - сброс состояния ошибок двигателя

Группа компонентов анализа телеметрии

- Индикатор Analyzing - горит зеленым, если в данный момент идет анализ телеметрии работы двигателя
- Слайдер Analyze - запуск анализа телеметрии работы двигателя
- Черное текстовое поле (Result) - результат анализа телеметрии

Для выполнения анализа нужно включить слайдер `Analyze` и запустить автоматический цикл системы с помощью кнопки `START` на панели `FLOW`. Функция `collect` соберет необходимую телеметрию двигателя и отправит её на анализ через MQTT-топик `node-red/request`. Результат анализа поступает через MQTT-топик `node-red/data` на текстовое поле `Result`

Flow dashboard

Панель автоматического запуска цикла работы системы (описан в разделе OpenPLC)

- Слайдер `Manual` - переключает режим управления, делает активными или неактивными другие кнопки управления
- `State` - индикатор состояния цикла работы (переменная состояния `CYRCLE_STATE %MW1` OpenPLC)
- Кнопка `Start` - запуск автоматического цикла

K1 dashboard

Панель мониторинга и контроля работы реле K1

- Индикатор состояния реле: красный - выключено, зеленый - включено
- Кнопки:
 - `Coil ON` - ручное включение реле

- Coil OFF - ручное выключение реле

AI-агент: Твой личный дежурный инженер на п8п

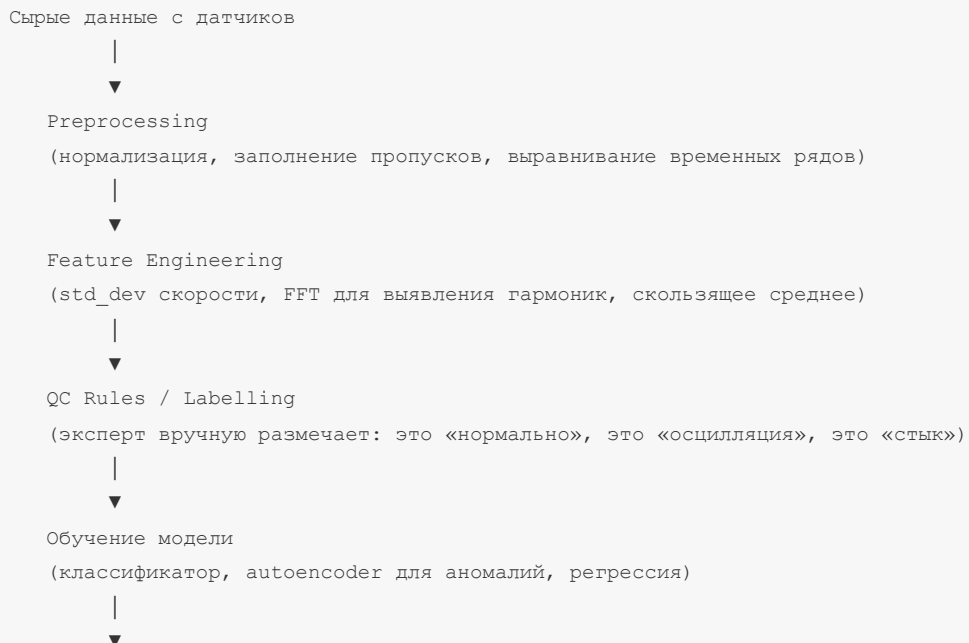
Звучит красиво и многообещающе. Но так ли это на самом деле. Как говорят: "It depends", то есть это зависит от многих факторов.

Представьте: у вас есть двигатель, управляемый по алгоритму FOC. Датчики каждую секунду шлют данные по CAN bus — скорость, токи d/q осей, температуру, параметры PID-регулятора. Двигатель работает, вращается, но периодически ведёт себя странно: то скорость скачет, то на целевое значение выходит с большим запозданием.

Как понять, что именно не так? И главное — как это сделать автоматически, не нанимая инженера, который будет сидеть и смотреть на графики?

Классический подход: ETL → ML → правила

В традиционном машинном обучении задача решается примерно так:



Инференс + пороговые правила
(если `anomaly_score > 0.7` → alert)

Объяснить с 🗨️

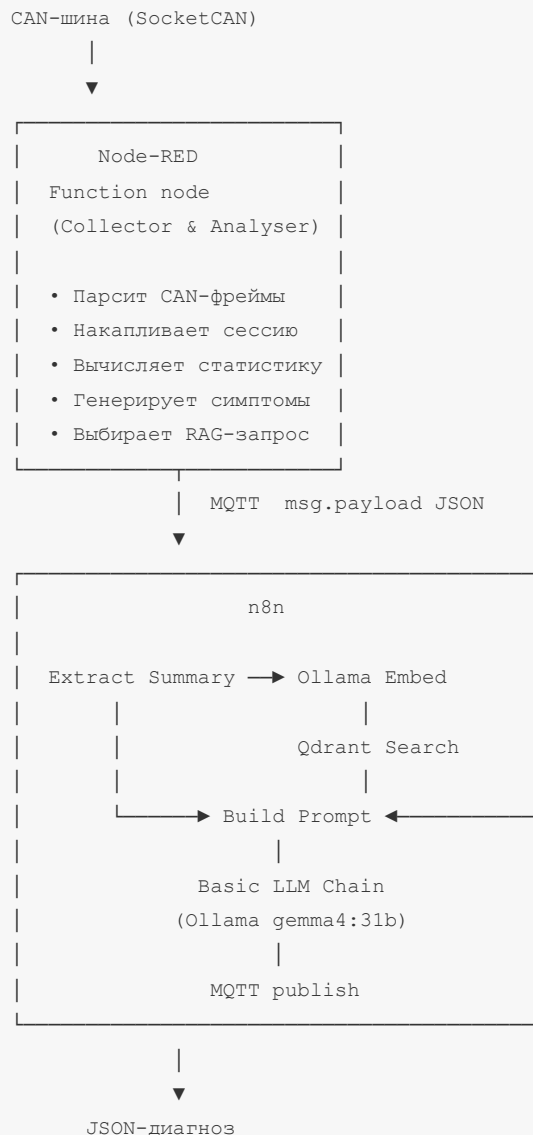
Это работает. Но у подхода есть очевидные издержки:

- Нужны **размеченные данные** — а их обычно нет, пока не накопишь достаточно аварий.
- Модель **не объясняет** себя: «аномалия с вероятностью 0.83» — это не диагноз.
- При изменении конфигурации двигателя или PID-коэффициентов **модель устаревает**.
- Весь пайплайн нужно поддерживать как отдельный проект: датасет, обучение, деплой.

Наш подход: стриминг → анализ на месте → LLM с контекстом

Мы пошли другим путём. Вместо обучения модели — инженерный здравый смысл на входе и языковая модель на выходе.

Общая архитектура




```
{state, diagnosis, root_cause,
 recommendations, confidence}
```

Объяснить с 🧑🏻💻

Что делает Node-RED: детектор событий

Ключевая идея первого звена: Node-RED не просто собирает данные — он уже анализирует их на лету.

Когда двигатель запускается, открывается сессия. Каждый входящий CAN-фрейм обновляет текущее состояние (`speed_rpm` , `iq` , `id` , `motor_state` , `pid_*`). Рядом с каждым снимком фиксируется и `target_rpm` — текущее задание со стороны оператора.

Когда двигатель возвращается в IDLE (или истекает тайм-аут), сессия *флашится*: вычисляется весь аналитический блок.

Хитрость с осевшими сэмплами

Самая частая ошибка при анализе телеметрии двигателя — считать среднее значение скорости `avg(speed_rpm)` по всей сессии. Если двигатель разогнался от 0 до 1000 об/мин, среднее будет ~500, и LLM решит, что мотор недотягивал до цели. На самом деле он просто разогнался.

Поэтому мы отделяем сэмплы переходного процесса от сэмплов установившегося режима:

```
settledSamples = runSamples
WHERE target_rpm == lastTargetRpm ← только последнее задание
AND speed_rpm >= 0.9 * lastTarget ← скорость ≥ 90% от цели
```

Объяснить с 🧑🏻💻

Только на этом подмножестве считается `settled_speed_avg` , `settled_speed_std` и ошибка регулирования. Рампа разгона не влияет на оценку качества регулятора.

Автоматический выбор RAG-запроса

Ещё один важный элемент: вместо того чтобы передавать LLM сырой JSON и просить его «придумай запрос к базе знаний» — Node-RED детерминированно выбирает поисковую фразу по правилам:

Условие	Запрос в Qdrant
Нет RUN-состояния	FOC startup failure alignment revup current
Пик Iq > 1 A	FOC current saturation anti-windup Iq limit
std_dev > 30 об/мин (в settled)	FOC speed loop PI oscillation gain tuning
Ошибка > 50 об/мин	FOC speed steady-state error PI gain integral
Нулевые момент и мощность	FOC zero torque Iq reference power loss
Норма	PMSM FOC speed control normal operation

Это короткая фраза без JSON-символов — её нельзя испортить даже если в данных есть странные значения.

Был испытан вариант с передачей LLM телеметрии и указанием - проанализировать и подготовить

запрос в RAG. Использовались различные модели от Llama, Qwen, Gemini flash. Но так и не удалось добиться стабильного результата.

Что делает n8n: RAG + LLM как эксперт без учителя

Второе звено берёт подготовленный Node-RED'ом пакет и добавляет контекст из базы знаний.

RAG — почему это может быть полезным

LLM хорошо рассуждает, но не знает конкретики вашего SDK. Документация STM32 Motor Control Workbench была заранее загружена в Qdrant (векторная БД).

Для этого с помощью Github copilot был составлен скрипт [ingest_rag.py](#).

Теперь при каждом диагнозе автоматически находятся 4-5 наиболее релевантных фрагмента — например, описание параметров `pid_speed_kp / ki` и что происходит при их неправильной настройке.

Модель получает не просто «`pid_speed_kp = 2730`», а ещё и инженерный контекст: когда этот параметр актуален, какую проблему он решает, что с ним делать.

Временной ряд в промпте

В промпт также вставляется таблица реального временного ряда (до 60 строк):

```
TIME SERIES — RUN state, 42 sample(s):
```

t_ms	target_rpm	speed_rpm	iq_A	id_A
0	100	132.0	0.0216	0.0017
499	100	60.0	0.0249	0.0054
999	1000	90.0	0.0308	0.0007
1499	1000	498.0	0.0244	-0.0042
1999	1000	870.0	0.0246	-0.0039
2499	1000	990.0	0.0206	-0.001
...				

[Объяснить с 🤖](#)

Это позволяет модели видеть динамику: как менялось задание, как реагировала скорость, как вёл себя ток.

Пример реального вывода

Входные данные — сессия 10 секунд, двигатель разогнался с 100 до 1000 об/мин и вышел на установившийся режим:

```
target_rpm          = 1000
settled_speed_avg    = 996.5    ← фактическая скорость в УР
settled_speed_std     = 4.2      ← почти нет осцилляций
steady_state_err_rpm = 3.5      ← ошибка < 0.4%
symptoms             = no anomalies detected
```

Объяснить с 

Ответ LLM:

```
{
  "state": "normal",
  "confidence": 0.92,
  "diagnosis": "Motor successfully ramped from 100 RPM to 1000 RPM target.
    Settled speed avg 996.5 RPM gives steady-state error of 3.5 RPM (0.35%),
    well within acceptable limits. settled_speed_std 4.2 RPM indicates
    stable regulation with no oscillation.",
  "root_cause": "No fault detected — normal operation.",
  "recommendations": [
    {
      "parameter": "pid_speed_kp",
      "current_value": 2730,
      "action": "monitor",
      "reason": "Current gains produce acceptable steady-state error;
        monitor under varying load conditions"
    }
  ]
}
```

Объяснить с 

Без исправления `settled_speed` — LLM видел бы `avg=626`, `std_dev=404` и писал про «нестабильность» и «недостаточное усиление ПИД».

Код, написанный с помощью промпта

Есть момент, который стоит упомянуть отдельно: часть кода в этом проекте написана не вручную. Функция-коллектор в Node-RED с парсингом CAN-фреймов, логикой сессий, вычислением `settledSamples` и генерацией симптомов — создана через [промпт](#) в Github copilot. Код обработки в `n8n` — узлы `Extract Telemetry Summary` и `Build Prompt` — тоже.

Это не значит «нажал кнопку и получил готовое». Это итеративный процесс: сформулировал требование, получил черновик, указал на ошибку, уточнил поведение на граничном случае, снова уточнил. Но принципиальная разница с написанием кода вручную — в том, где уходит время инженера. Модель пишет черновик. Инженер проверяет логику, находит, что порог 90% не работает при смене цели в середине сессии — и уточняет: снимок `target_rpm` должен быть в каждом

сэмпл, а `lastTargetRpm` берётся из последней записи в `targetHistory`. Модель исправляет. Ещё одна итерация.

Время на «написать код» сокращается в разы. Время на думать, что код должен делать — остаётся прежним. И это правильное распределение усилий.

Промпт как документация

Есть и практический бонус: промпт, по которому сгенерирован код, сам по себе является спецификацией. Именно поэтому в этом проекте рядом с кодом лежат файлы `node-red-collect.md` и `n8n-PMSM_Diagnostic.md` — не просто документация, а полные промпты, по которым можно воссоздать функции с нуля. Если через полгода нужно будет изменить логику анализа или добавить новый тип CAN-фрейма — берёшь промпт, вносишь изменение в требование и регенерируешь нужный фрагмент.

Где это не работает

Промпт-генерация хороша для структурированного шаблона и обработки данных с чёткими правилами. Она хуже справляется с задачами, где правильный ответ неочевиден самому инженеру — например, выбор оптимальных порогов для детекции осцилляций или архитектурные решения с долгосрочными последствиями. Там по-прежнему нужен думающий человек. Модель — инструмент, а не замена экспертизе.

Что можно улучшить.

Адаптивный порог для анализа

Сейчас считается, что двигатель «вышел в режим», когда скорость $\geq 90\%$ от цели. Это фиксированный порог. Для быстрых высокоинерционных нагрузок он слишком мягкий, для лёгких — слишком жёсткий. Хорошим улучшением было бы вычислять порог динамически — например, на основе замеренного времени разгона предыдущих сессий.

Накопление истории сессий

Сейчас каждая сессия анализируется независимо. Если бы Node-RED или n8n сохранял историю (хотя бы последние 50 сессий в простую БД — SQLite, InfluxDB), LLM мог бы видеть тренд: «за последние 3 дня `settled_std` стабильно растёт — деградация подшипника или усиление нагрузки». Сейчас такое сравнение недоступно.

Обратная связь по диагнозу

LLM может ошибиться. Прямо сейчас результат уходит в MQTT и нигде не проверяется. Можно добавить в Node-RED простую UI-панель с кнопками «диагноз верный / неверный». Эти оценки копятся и периодически обогащают базу знаний в Qdrant — фактически RAG улучшается от работы с реальными данными без переобучения.

Несколько LLM для консенсуса

Модель `gemma4:31b` справляется хорошо, но у языковых моделей бывают "галлюцинации". Для критичных случаев (`state=unstable`, `confidence < 0.7`) можно отправлять тот же промпт дополнительной модели и сравнивать `state`. Если они не согласны — фиксировать как «спорный диагноз» и отправлять инженеру.

Итог

Основная идея проста: дайте инструменту правильно подготовленные данные, и он объяснит вам, что происходит, без предварительного обучения на исторических авариях.

Node-RED здесь — не просто шина данных. Это уровень инженерной интерпретации: он знает, что такое переходный процесс, что такое установившийся режим, и не путает одно с другим. n8n добавляет документацию через RAG. LLM связывает всё в читаемый диагноз.

Такой стек можно поднять локально на обычном сервере или использовать облачные ресурсы. LLM используется как облачный ресурс. А сервисы Qdrant, Node-RED и n8n отлично работают на Orange pi.

Исходный код проекта Node-red, n8n находится в [этом](#) репозитории.

Теги: diy-проекты, orangepi, raspberrypi, llm-модели, n8n, node-red, llama

Хабы: DIY или Сделай сам, Прототипирование, Программирование микроконтроллеров, Облачные сервисы, Искусственный интеллект



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



16K+

61

Охват за 30 дней

Карма

Пирогов Владислав @vladipirogov

Инженер по автоматизированным системам управления

Подписаться



Поток DIY доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для всех

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

[Перейти в поток DIY](#)

Комментировать

Публикации

[ЛУЧШИЕ ЗА СУТКИ](#)

[ПОХОЖИЕ](#)



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

 Простой  4 мин  16K

 +96

 30

 27

 **interpres**
18 часов назад

Электроинструмент становится хуже, и это делается намеренно

 Простой  11 мин  27K

Обзор

Перевод

 +71

 45

 83

 **prohronus**
19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

 5 мин  9.3K

Кейс

 +51

 9

 9

 **alizar**
22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

 Простой  7 мин  12K

Обзор

 +37

 59

 18

 **AlexSerbul**
23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

 Средний  14 мин  9.9K

Мнение

 +28

 33

 11

 **TrexSelectel**
19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

 5 мин  8.8K

Обзор

 +27

 10

 0

 **mariklolik**
18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

 Средний  7 мин  7.5K

Кейс

 +23

 23

 6



nick_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

 5 мин  7.1K

Recovery Mode

 +22

 8

 5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

 11 мин  7K

 +17

 9

 7



infgotoinf

10 часов назад

Perl — зря забытый язык программирования?

 11 мин  8.5K

Из песочницы

 +16

 13

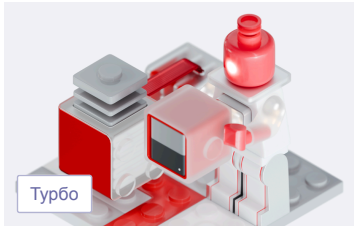
 9

Что нам стоит IaaS построить: как мы собирали прочный фундамент для облака

Турбо

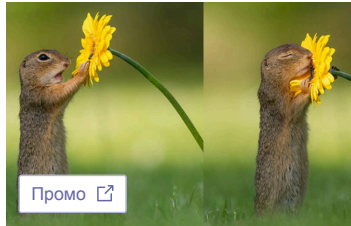
Показать еще

МИНУТОЧКУ ВНИМАНИЯ



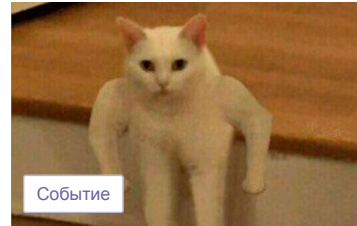
Турбо

Собери облачный пазл и выиграй призы



Промо

Запахло весной и скидками в Промокодусе



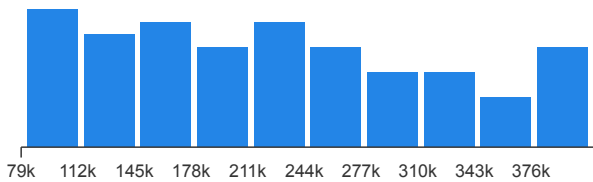
Событие

Качаем к лету хард- и софт-скиллы на айтишных ивентах

СРЕДНЯЯ ЗАРПЛАТА В IT

221 623 ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 32 194 анкет, за 1-ое пол. 2026 года. Проверьте «в рынке» ли ваша зарплата или нет!



[Проверить свою зарплату](#)

ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 27K  83 

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 13K  14 

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 7K  15 

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

 5.5K  11 

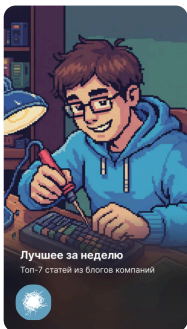
+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

 14K  10 

Что нам стоит IaaS построить: как мы собирали прочный фундамент для облака

Турбо

ИСТОРИИ



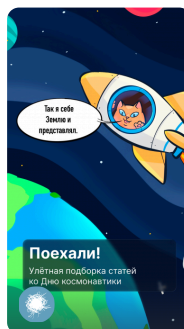
Годнота из блогов компаний



Только 10% решат эту головоломку



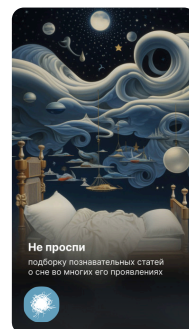
Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

БЛИЖАЙШИЕ СОБЫТИЯ

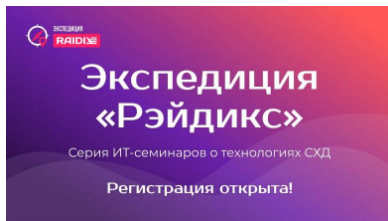


2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная
премия, учрежденная в
2000 году**

Санкт-Петербург

Маркетинг



3 марта – 9 июня

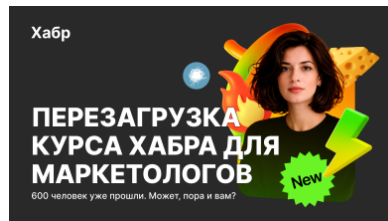
Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону

Разработка

Другое

Больше событий в календаре



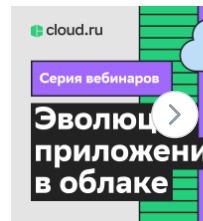
9 марта – 1 июня

**Всё, что нужно, чтобы
стартовать продвижение
бренда на Хабре**

Онлайн

Маркетинг

Больше событий в календаре



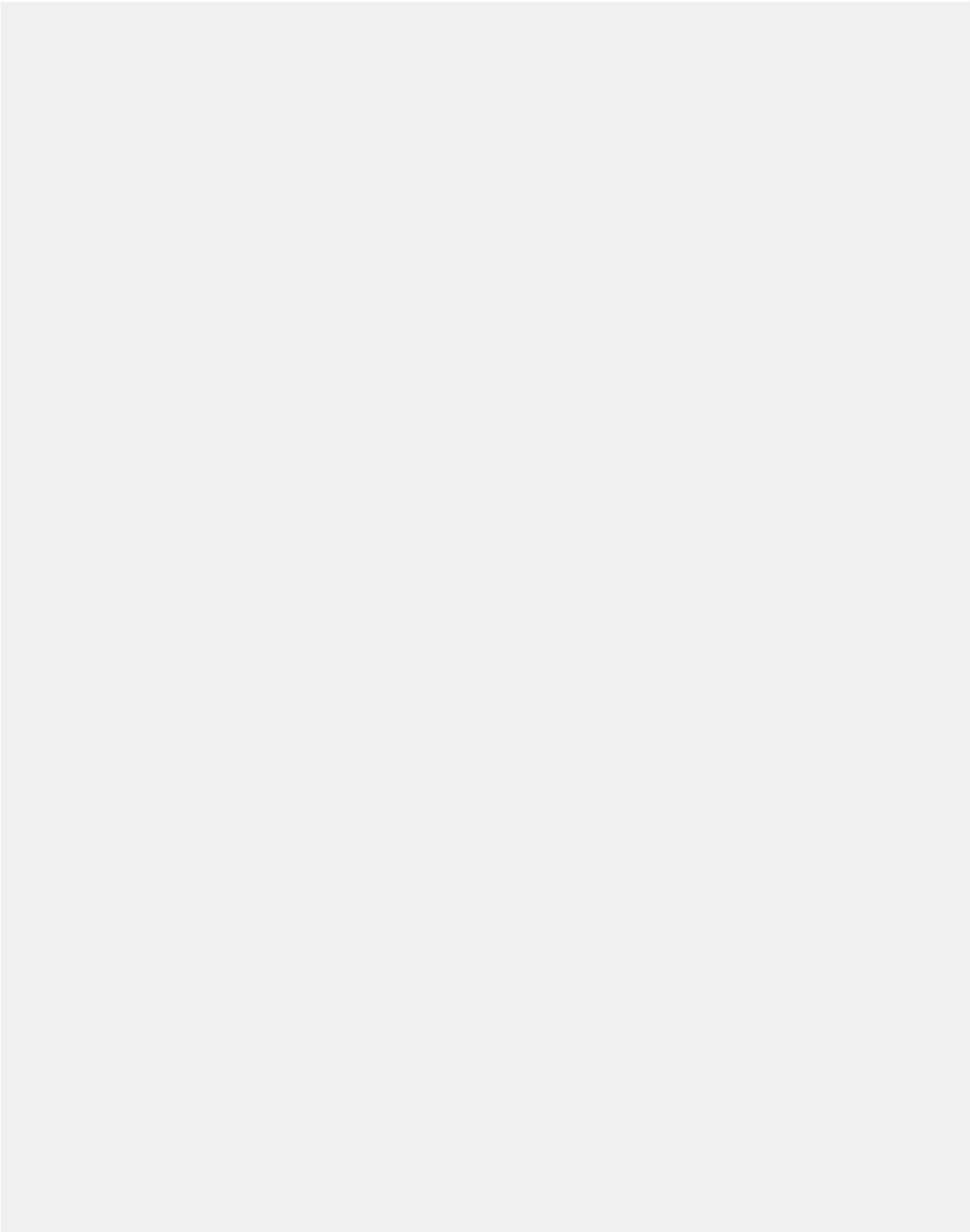
19 марта – 28 апреля

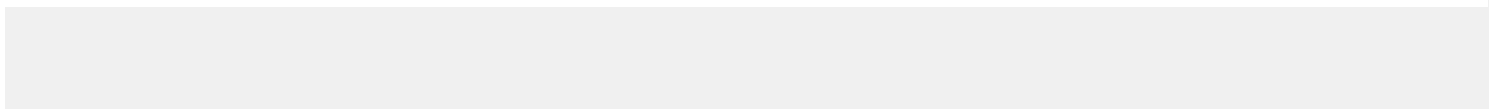
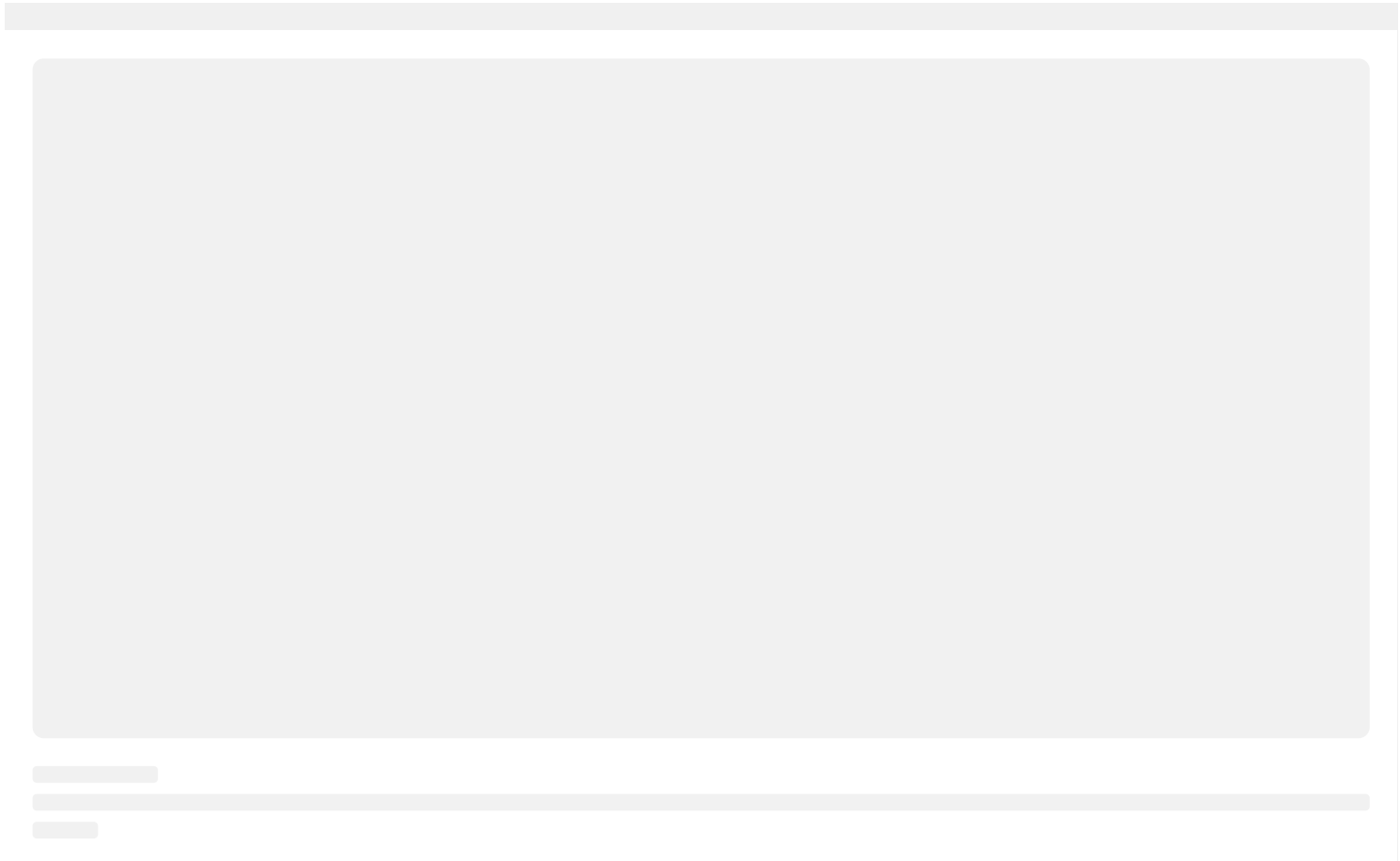
**Серия вебинаров
«Эволюция приложений
в облаке»**

Онлайн

Разработка

Больше событий в календаре





Ваш аккаунт	Разделы	Информация	Услуги
Войти	Статьи	Устройство сайта	Корпоративный блог
Регистрация	Новости	Для авторов	Медийная реклама
	Хабы	Для компаний	Нативные проекты
	Компании	Документы	Образовательные программы
	Авторы	Соглашение	Стартапам
	Песочница	Конфиденциальность	
   			
Настройка языка			
Техническая поддержка			
© 2006–2026, Habr			